

Aprendizaje de Máquina

U2 Análisis de Clústeres

Héctor Maravillo

Section 1

Graph Theory

Basic Definitions from Graph Theory

A **graph** G is defined as an ordered pair $G = (V, E)$, where $V = \{v_1, \dots, v_N\}$ is a set of **vertices** and E is a set of **edges** connecting some pairs of vertices.

An edge, connecting the vertices v_i and v_j , will be denoted either by e_{ij} or by (v_i, v_j) .

- When the ordering of v_i and v_j is of no importance, then we deal with **undirected graphs**.
- Otherwise, we deal with directed graphs.
- If no cost is associated with the edges of the graph, we deal with **unweighted graphs**.
- Otherwise, we deal with **weighted graphs**.
- A graph is **complete** if all its vertices are connected to each other by an edge.

Basic Definitions from Graph Theory

- A **path** in G , between vertices v_{i_1} and v_{i_n} , is a sequence of vertices and edges of the form

$$v_{i_1} e_{i_1, i_2} v_{i_2} \dots v_{i_{n-1}} w_{i_{n-1}, i_n} v_{i_n}$$

- If in this path v_{i_1} coincides with v_{i_n} , the path is called a **loop**. In the special case in which an edge connects a vertex to itself, we have a **self-loop**.

Basic Definitions from Graph Theory

- A **path** in G , between vertices v_{i_1} and v_{i_n} , is a sequence of vertices and edges of the form

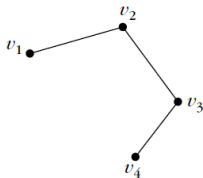
$$v_{i_1} e_{i_1, i_2} v_{i_2} \dots v_{i_{n-1}} w_{i_{n-1}, i_n} v_{i_n}$$

- If in this path v_{i_1} coincides with v_{i_n} , the path is called a **loop**. In the special case in which an edge connects a vertex to itself, we have a **self-loop**.
- If no vertex or edge is repeated, it is called a **cycle**.

Basic Definitions from Graph Theory

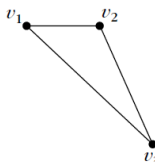
- A **subgraph** $G' = (V', E')$ of G is a graph with $V' \subset V$ and $E' \subset E$, where all the ends of an edge $e \in E'$ are in V' .
- A subgraph $G' = (V', E')$ is **connected** if there exists at least one path connecting any pair of vertices in V' .
- A **maximally connected** subgraph of G is a connected subgraph G' of G that contains as many vertices of G as possible.
- A **maximally complete** subgraph is a complete subgraph G' of G that contains as many vertices of G as possible.

Basic Definitions from Graph Theory



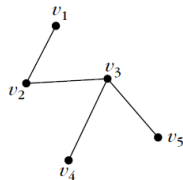
Path

(a)

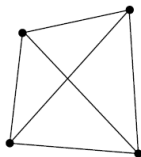
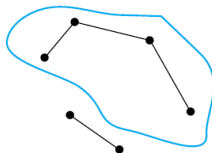
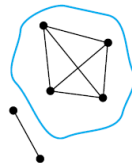


Loop

(b)

Connected
graph

(c)

Complete
graphMaximally
connected
subgraphMaximally
complete
subgraph

Basic Definitions from Graph Theory

- The **node connectivity** of a connected subgraph is the largest integer k such that all pairs of nodes are joined by at least k paths having no nodes in common.
- The **edge connectivity** of a connected subgraph is the largest integer k such that all pairs of nodes are joined by at least k paths having no edges in common.
- The **degree** of a connected subgraph is the largest integer k such that each node has at least k incident edges.

Basic Definitions from Graph Theory

- A **spanning tree** is a connected graph (containing all the vertices of the graph) and having no cycles (that is, there exists only one path connecting any two pairs of nodes in the graph). If the edges of the graph are weighted, we define as the weight of the spanning tree the sum of the weights of its edges.
- A **minimum spanning tree (MST)** is a spanning tree with the smallest weight among all spanning trees connecting the nodes of the graph.¹

¹Note that there may be more than one minimum spanning trees for a given graph. However, when the weights of the edges of G are different from each other, then the *MST* is unique.

Section 2

Graph-based Agglomerative Clustering

Graph-based Agglomerative Clustering

The general agglomerative scheme in the context of graph theory is known as the **graph theory-based algorithmic scheme (GTAS)**.

The algorithm follows exactly the same iteration steps as the **generalized agglomerative scheme (GAS)**, with the exception of step 2.2. This is expressed as

$$g_h(C_i, C_j) = \begin{cases} \min_{r,s} g_h(C_r, C_s) & \text{for dissimilarity functions} \\ \max_{r,s} g_h(C_r, C_s) & \text{for similarity functions} \end{cases}$$

Graph-based Agglomerative Clustering

The **proximity function** $g_h(C_r, C_s)$ between two clusters is defined in terms of

- A **proximity measure** between vectors (that is nodes in the graph)
- A certain **constraints** imposed by the property h on the subgraphs that are formed.

In a more formal way, g_h is defined as

$$g_h(C_r, C_s) = \min_{u \in C_r, v \in C_s} \{d(u, v) : \text{the induced subgraph } G[C_r \cup C_s] \text{ is connected and either holds some property}\}$$

Graph-based Agglomerative Clustering

The most common algorithms of this type are:

- **Single Link Algorithm:**

$$g_h(C_r, C_s) = \min_{x \in C_r, y \in C_s} d(x, y)$$

Here **connectedness** is the only prerequisite.

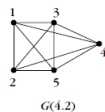
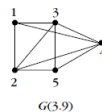
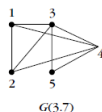
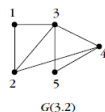
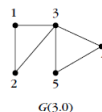
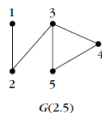
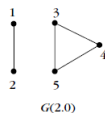
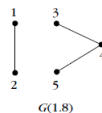
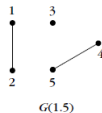
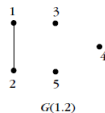
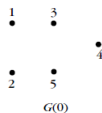
- **Complete Link Algorithm:**

$$g_h(C_r, C_s) = \max_{x \in C_r, y \in C_s} d(x, y)$$

The only prerequisite here is that of **completeness**

Example

$$P = \begin{bmatrix} 0 & 1.2 & 3 & 3.7 & 4.2 \\ 1.2 & 0 & 2.5 & 3.2 & 3.9 \\ 3 & 2.5 & 0 & 1.8 & 2.0 \\ 3.7 & 3.2 & 1.8 & 0 & 1.5 \\ 4.2 & 3.9 & 2.0 & 1.5 & 0 \end{bmatrix}$$



Section 3

Minimum Spanning Tree Algorithms

Minimum Spanning Tree Algorithms

This algorithm is based on the idea of the **Minimum Spanning Tree (MST)** and is motivated by the way human perception works.

More precisely, humans organize information with the **most economical encoding**. For example, the more likely way for a human to organize the points in Figure is in four groups (clusters).

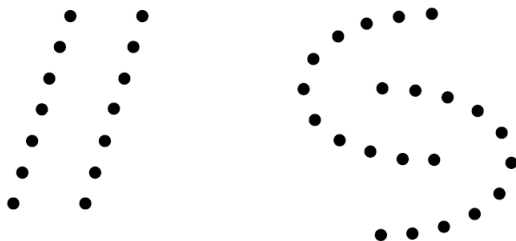


Figure: Example

Minimum Spanning Tree Algorithms

Let us consider the **complete graph** G , each node of which corresponds to a point of X .

The **weight of an edge** $e = (x_i, x_j)$, connecting two nodes x_i and x_j , is set equal to $d(x_i, x_j)$ of the corresponding points in the feature space

$$w(e) = d(x_i, x_j)$$

Also, we say that two edges e_1 and e_2 are k **steps away** from each other if the minimum path that connects a vertex of e_1 and a vertex of e_2 has length equal to $k - 1$, that is, **contains** $k - 1$ **edges**.

[Another way to look at it is to count the number of edges in the minimum path between two vertices of e_1 and e_2 and add one to it.]

Inconsistent edges

The idea of the algorithm is the following: determine the minimum spanning tree of G and then **remove** the edges that are **unusually large** compared with their neighboring edges.

These edges are called **inconsistent**, and it is expected that they connect points from different clusters.

Inconsistent edges

For each edge e , we consider **all the edges**, e_i , that lie k steps away from it, at the most, and we compute the **mean**, m_e , and the **standard deviation**, σ_e , of their weights.

If w_e lies more than q (typically $q = 2$) standard deviations (σ_e) away from m_e , then we consider e as **inconsistent**.

Formally, the edge e is **inconsistent** if one of the following conditions is met:

$$w(e) > m_e + q\sigma_e$$

$$w(e) < m_e - q\sigma_e$$

From this, it is clear that the characterization of an edge as inconsistent is somewhat **subjective** and depends on k and q , which are preselected.

The MST Clustering Algorithm

- Construct a **complete graph** G such that:
 - Its vertices correspond to the vectors of X .
 - $w(x_i, x_j) = d(x_i, x_j)$, $i, j = 1, \dots, N$, $i \neq j$
- Determine the *MST* of G .
- Identify the **inconsistent** edges of the *MST*.
- The clusters are the **connected components** of the *MST* after the removal of the inconsistent edges.

Exercise

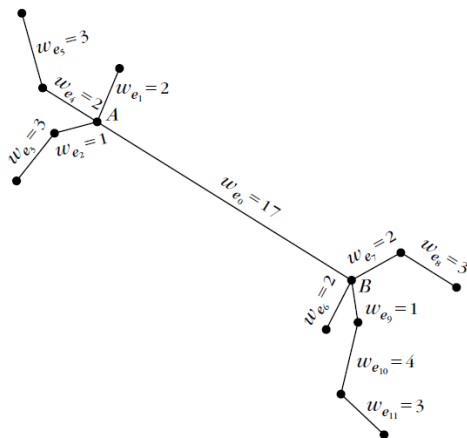


Figure: The *MST* of a graph. The edge e_0 is inconsistent, while e_{11} is consistent

Exercise

Let $k = 2$ and $q = 3$.

The edges lying two steps at the most from e_0 are $e_1, \dots, e_{10}$. The mean m_{e_0} and the standard deviation σ_{e_0} , corresponding to e_0 are 2.3 and 0.9, respectively. Thus, w_{e_0} lies 16.3 standard deviations (σ_{e_0}) away from m_{e_0} . Therefore, e_0 is an **inconsistent edge**.

Let us now consider the edge e_{11} . Working as before, we find that its neighbors are e_{10} and e_9 , so $m_{e_{11}}$ and $\sigma_{e_{11}}$ are 2.5 and 1.50, respectively. Thus $w_{e_{11}}$ is 0.24 standard deviations $\sigma_{e_{11}}$ away from $m_{e_{11}}$. Therefore, e_{11} is not an inconsistent edge.

Generally, the condition is that there is no other point $x \in X$ different from x_i and x_j within a region $R(x_i, x_j)$ defined in terms of the distance $d(x_i, x_j)$.

Remarks

- This algorithm works satisfactorily for many cases where the clusters are **well separated**.
- Some suggestions for the use of the *MST* algorithm for the cases where we have **touching clusters**, as well as for the case where the clusters have **different densities**. However, they implicitly require knowledge of the shape of the clusters.
- This algorithm **does not depend** on the **order** in which data are considered by the algorithm and, also, **no initial conditions** are required.

Section 4

Algorithms based on Regions of Influence

Algorithms based on regions of influence

An extension of the MST involves **regions of influence** for each pair of vectors of X .

This idea was used by many researchers in order to overcome the problem associated with the MST algorithms.

Algorithms based on regions of influence

Let us consider two distinct vectors $x_i, x_j \in X$. Their **region of influence** is defined as

$$R(x_i, x_j) = \{x : \text{cond}(d(x, x_i), d(x, x_j), d(x_i, x_j)), x_i \neq x_j\}$$

where

$$\text{cond}(d(x, x_i), d(x, x_j), d(x_i, x_j))$$

is a condition among the distances $d(x, x_i), d(x, x_j), d(x_i, x_j)$.

Different choices of *cond* give rise to different shapes of regions of influence.

Regions of influence

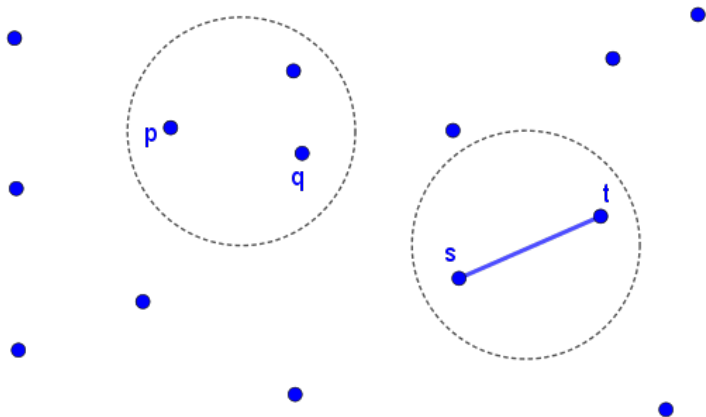


Figure: Condition $(p, q) \in E \Leftrightarrow R(p, q) \cup (X - \{p, q\}) = \emptyset$.

Regions of influence

Examples of conditions:

- **Gabriel graph GG**

$$d^2(x, x_i) + d^2(x, x_j) < d^2(x_i, x_j)$$

- **Relative Neighborhood Graph RNG**

$$\max\{d(x, x_i), d(x, x_j)\} < d(x_i, x_j)$$

Algorithm based on Regions of Influence

- Initialize $G = (V, E)$ with $V = X$ and $E = \emptyset$.
- For $i = 1$ to N
 - For $j = i + 1$ to N
 - Determine the region of influence $R(x_i, x_j)$
 - If $R(x_i, x_j) \cup (X - \{x_i, x_j\}) = \emptyset$ then

$$E = E \cup \{(x_i, x_j)\}$$

- Determine the connected components of the G and identify them as clusters.

In words, the edge between x_i and x_j is added if no other vectors of X lies $R(x_i, x_j)$. This is because when x_i and x_j are closely located, it is expected that no other points of X will be in $R(x_i, x_j)$.

References

- Theodoridis, S. & Koutroumbas, K. *Pattern Recognition*, 4th ed., Academic Press, 2009